LOCAL-FIRST AI

RUST CORE

MLX.PILOT

Plataforma de **Execução e Automação** de Modelos de Linguagem (LLMs).

CONTEXTO ATUAL

O Problema da IA Centralizada

Desafios técnicos e econômicos que motivaram o desenvolvimento da arquitetura local-first.

ERR_DEPENDENCY

Dependência Excessiva

Acesso a LLMs restrito a APIs proprietárias (OpenAI, Anthropic), criando "vendor lock-in" e vulnerabilidade a instabilidades de serviço externo.

ERR_HIGH_COST

Custos Elevados

Modelos de cobrança por token tornam inviável a experimentação acadêmica em larga escala e o uso contínuo em aplicações intensivas.

ERR_PRIVACY

Privacidade e Controle

Envio obrigatório de dados sensíveis para servidores de terceiros (Black Box), impossibilitando o uso em contextos de sigilo ou dados médicos/pessoais.

ERR_NO_HARDWARE

Experimentação Limitada

Dificuldade em testar arquiteturas customizadas, quantizações específicas (GGUF/AWQ) e otimizações de hardware local sem ferramentas adequadas.

SOLUTION_OVERVIEW

A PROPOSTA MLX-PILOT

Uma plataforma desktop **Local-First** projetada para devolver o controle, reduzir custos e habilitar a pesquisa avançada em IA.

01

EXECUÇÃO LOCAL

Capacidade de rodar LLMs (Llama 3, Mistral, Gemma) diretamente no hardware do usuário, garantindo privacidade total dos dados e zero latência de rede.

STATUS: OFFLINE-READY

02

MULTI-BACKEND

Arquitetura agnóstica que unifica **MLX**, **llama.cpp** e **Ollama** sob uma única interface e API, permitindo flexibilidade total de infraestrutura.

STATUS: UNIFIED_API

03

EXPERIMENTAÇÃO

Sandbox controlado para estudantes e pesquisadores testarem hiperparâmetros, quantizações e arquiteturas de modelos sem custos de API.

STATUS: ACADEMIC_MODE

04

LABORATÓRIO PRÁTICO

Transforma qualquer computador com GPU/NPU em uma estação de trabalho de IA, servindo como base para disciplinas de Computação e TCCs.

STATUS: LAB_INSTALLED

SYSTEM ARCHITECTURE

Visão Arquitetural

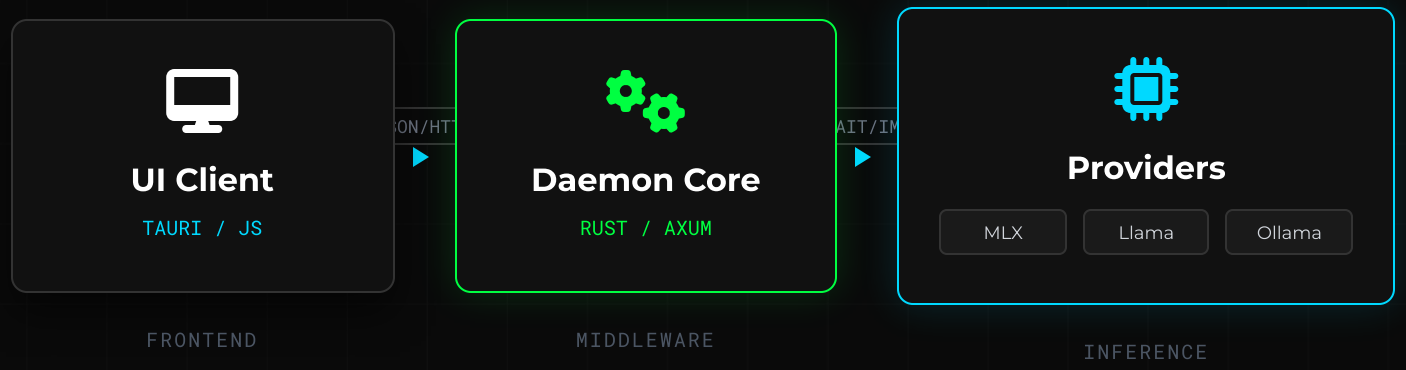
CORE::RUST // UI::TAURI // PROVIDERS::MODULAR

Rust Core
Alta performance e memory safety

Daemon HTTP
Axum + Tokio (Async Runtime)

Sistema Modular
Arquitetura baseada em Providers

UI Desktop
Tauri (Webview nativa leve)



SYSTEM ONLINE

MULTI-PROVIDER ARCHITECTURE

Backends Suportados

PROVIDER_SELECTION_MODE = "AUTO"

MLX

APPLE SILICON NATIVE

Framework de arrays otimizado para arquitetura Apple, permitindo acesso direto à memória unificada.

METAL API | UNIFIED MEMORY

- Otimização nativa para chips M1/M2/M3
- Lazy evaluation para alta performance
- Suporte a modelos Fine-tuned localmente

NPU UTILIZATION HIGH

llama.cpp

CROSS-PLATFORM CORE

Motor de inferência C++ puro focado em máxima eficiência e compatibilidade de hardware.

GGUF FORMAT | CPU + GPU

- Quantização Avançada (4-bit, 8-bit)
- Suporte Híbrido (Offload parcial p/ GPU)
- Execução em hardware legado (AVX2)

COMPATIBILITY MAX

Ollama

SERVICE INTEGRATION

Camada de serviço que simplifica o gerenciamento de modelos e exposição via API.

REST API | MODEL MANAGER

- Download e gestão automática de Blobs
- Interface unificada para múltiplos modelos
- Fácil integração com ferramentas externas

EASE OF USE HIGH

CORE FEATURES

Diferenciais Técnicos

SYSTEM_CAPABILITIES::V1.0 // INNOVATION_MODULE

LIVE DEMO FEATURES

Demonstração Funcional

CURRENT_BUILD: v0.8.2-alpha

Chat com Streaming

REAL-TIME TOKEN GENERATION

Interface de chat responsiva utilizando Server-Sent Events (SSE) para exibir tokens assim que são inferidos, sem latência perceptível.

```
User: Explique Rust em uma frase.
Bot: Rust é uma linguagem de sistemas focada em segurança e concorrência.
TOK/S: 45.2                                          LATENCY: 12ms
```

Model Manager

HUGGING FACE INTEGRATION

Catálogo integrado para busca e download direto de modelos (GGUF/MLX) com verificação automática de hash e compatibilidade.

```
Llama-3-8B-Instruct-v0.2.Q4_K_M.gguf                 3.8GB / 5.2GB
SOURCE: HF_HUB                                       DOWNLOADING...
```

Seleção de Backend

HOT-SWAPPABLE PROVIDERS

Capacidade de alternar dinamicamente entre motores de inferência (MLX, Llama.cpp, Ollama) sem reiniciar a aplicação.

```
Select Provider:
MLX (Apple)   llama.cpp   Ollama
> Provider switched successfully.
> Loaded 8B parameters in VRAM.
```

Execução Local

PRIVACY & OFFLINE FIRST

Ambiente totalmente isolado que garante que nenhum dado do chat ou telemetria deixe a máquina local do usuário.

```
Network Status                                       DISCONNECTED
  Local Server: Port 3000
0 External Requests Blocked
```

ACADEMIC GOALS

Objetivo Acadêmico

RESEARCH_FOCUS // EXPERIMENTATION // BENCHMARK

01

ARQUITETURA MODULAR

Estudo aprofundado sobre desacoplamento de sistemas, separando interface (Tauri) do controle lógico (Rust) e inferência.

02

LOCAL VS REMOTO

Análise comparativa crítica entre execução local (privacidade, latência zero) e nuvem (custo, escalabilidade).

03

BENCHMARK

Coleta sistemática de métricas de performance: Tokens por segundo (TPS), Tempo para o primeiro token (TTFT) e uso de memória.

04

STRESS TEST GPU

Avaliação de limites de hardware com modelos massivos (7B+), monitorando

05

BASE DE PESQUISA

Criação de uma plataforma estável e documentada para servir como fundação

PARCERIA ESTRATÉGICA

Alinhamento com NVIDIA

CUDA_ACCELERATION_ENABLED = TRUE

RTX 4090 | A100 TENSOR

acima de 7B parâmetros (Llama-3-70B, Mixtral 8x7B) utilizando aceleração de hardware dedicada.

Análise de VRAM

Monitoramento granular do consumo de memória de vídeo durante carregamento de camadas (offloading) e contexto estendido.

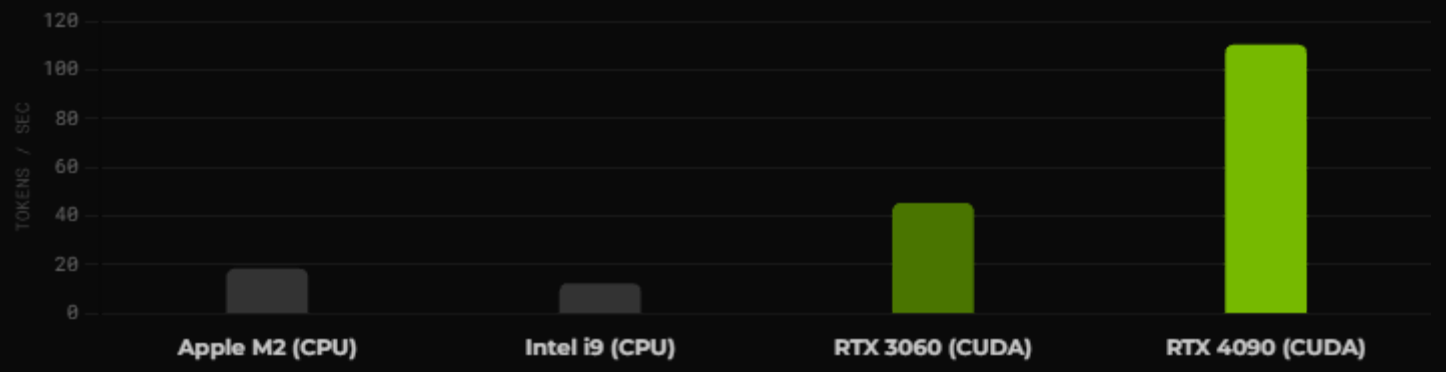
CUDA vs CPU

Benchmark comparativo de throughput (tokens/s) entre execução puramente em CPU e pipelines acelerados via CUDA.

Validação Acadêmica

INFERENCE SPEED (TOKENS/S)

Model: Llama-3-8B-Instruct-Q4



VRAM OPTIMIZATION

24GB vRAM

Suporta quantização Q4_K_M de modelos até 30B

TIME TO FIRST TOKEN

45ms avg

▼ 120ms vs CPU

Latência reduzida via Kernel Fusion

PROJECT MANAGEMENT

Backlog Atual

SPRINT: CONSOLIDAÇÃO // PI_III

● High Priority ● Medium Priority

BACKLOG / FUTURO 4

BENCHMARK TICKET-101

Comparativo de Performance

Analysis

GPU TICKET-102

Stress Test VRAM

SECURITY TICKET-105

Hardening do Daemon

SPRINT ATUAL (ALTA PRIORIDADE) 4

RUST CORE #ACTIVE

Estrutura Modular em Rust

Refatoração do monorepo para workspaces e traits de providers.

DV Compiling...

AXUM API #STABLE

Daemon HTTP Estável

Implementação de endpoints REST, tratamento de erros e CORS.

v0.5.2-alpha

LOGIC #INTEGRATION

Multi-provider Funcional

Switch dinâmico entre MLX e llama.cpp em tempo de execução.

CONCLUÍDO 2

SETUP DONE

~~Configuração Ambiente~~

DESIGN DONE

~~Protótipo UI Figma~~

RESEARCH & DEVELOPMENT

Expansão Acadêmica

PRIORITY_LEVEL: MEDIUM // NEXT_SPRINT

PLANNING_PHASE

PRIORIDADE MÉDIA

Benchmark Comparativo

Execução de baterias de testes padronizados para medir TPS (Tokens Per Second) e latência entre diferentes backends (MLX vs Llama.cpp) e hardwares.

METRICS_COLLECTION →

PRIORIDADE MÉDIA

Stress Test GPU

Simulação de carga máxima em VRAM com modelos de parâmetros elevados (>13B) para validar estabilidade térmica e gerenciamento de memória.

HARDWARE_LIMITS →

PRIORIDADE MÉDIA

Observabilidade

Implementação de sistema robusto de logs estruturados e métricas em tempo real para monitorar o comportamento do daemon e inferências.

TELEMETRY_SYSTEM →

PRIORIDADE MÉDIA

Hardening de Segurança

Reforço das camadas de segurança, incluindo sanitização rigorosa de inputs no chat e isolamento (sandbox) do ambiente de execução de código.

SECURITY_AUDIT →

FUTURE ROADMAP

Evolução

🏛 INSTITUTIONAL VALUE

Impacto Acadêmico

EDUCATION // RESEARCH // INNOVATION

Laboratório Prático de IA

REQ: GPU/CPU

Ambiente controlado para alunos experimentarem com LLMs sem custos de nuvem. Permite "botar a mão na massa" com inferência, prompts e fine-tuning básico.

Base para TCCs

DOCS: READY

Framework extensível que serve de ponto de partida para monografias. Alunos podem focar em desenvolver novos plugins ou agentes ao invés de configurar infraestrutura.

Uso em Disciplinas

COURSE: AI_101

Ferramenta didática para aulas de Inteligência Artificial, Sistemas Distribuídos e Compiladores, demonstrando conceitos teóricos na prática.

Plataforma Institucional

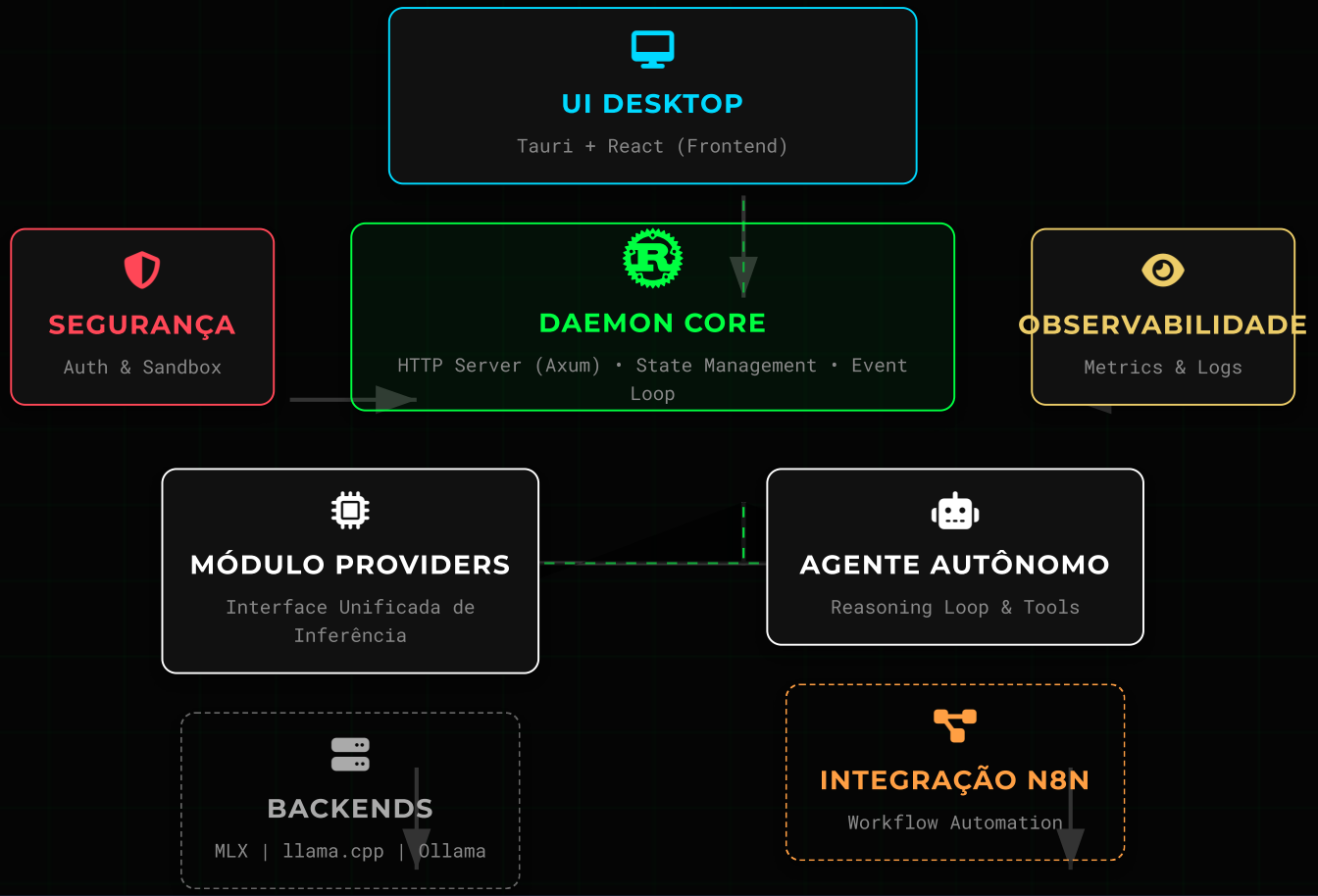
SCOPE: CAMPUS

Hub centralizado para pesquisa e desenvolvimento dentro da universidade, fomentando grupos de estudo e parcerias com empresas (ex: NVIDIA).

SYSTEM ARCHITECTURE V2.0

Arquitetura Futura

NEXT_GEN // AGENTIC_WORKFLOWS



Presentation · Core Logic · Modules · External/Integration

ARCHITECTURE_DIAGRAM // SCALE_READY

TECH STACK

Tecnologias Utilizadas

HIGH_PERFORMANCE // MEMORY_SAFE // LOCAL_FIRST

CORE FOUNDATION

v1.75+

Rust

Linguagem principal garantindo segurança de memória, performance nativa e zero-cost abstractions.

Core Logic | Daemon

v2.0-beta

Tauri

Framework para construção de UI desktop leve e segura utilizando tecnologias web (HTML/JS/CSS).

Frontend | WebView

v0.7

Axum

Servidor HTTP ergonômico e modular para comunicação robusta entre UI e Daemon via API REST/SSE.

API REST | Handlers

Runtime

Tokio

Runtime assíncrono para gerenciar concorrência, I/O não bloqueante e tarefas em background.

Async | Concurrency

INFERENCE ENGINES

MLX

APPLE RESEARCH

Framework de array otimizado para Apple Silicon, permitindo inferência de alta performance em Macs (M1/M2/M3).

Metal API

llama.cpp

GGML / GGUF

Motor de inferência C++ puro com suporte a quantização (GGUF), rodando eficientemente em CPU e GPU variadas.

Cross-platform

Ollama

MODEL RUNNER

Interface de alto nível para gerenciamento simplificado de modelos locais e compatibilidade de API.

Easy Setup

BENCHMARKS

Métricas e Performance

HARDWARE: APPLE M3 MAX (128GB RAM) // CUDA: N/A // METAL: ENABLED

BEST: MLX

62.4 tok/s

PEAK THROUGHPUT

Model: Llama-3-8B-Instruct (4-bit)

LOWEST: LLAMA.CPP

12ms

P50 LATENCY

18ms

P95 LATENCY

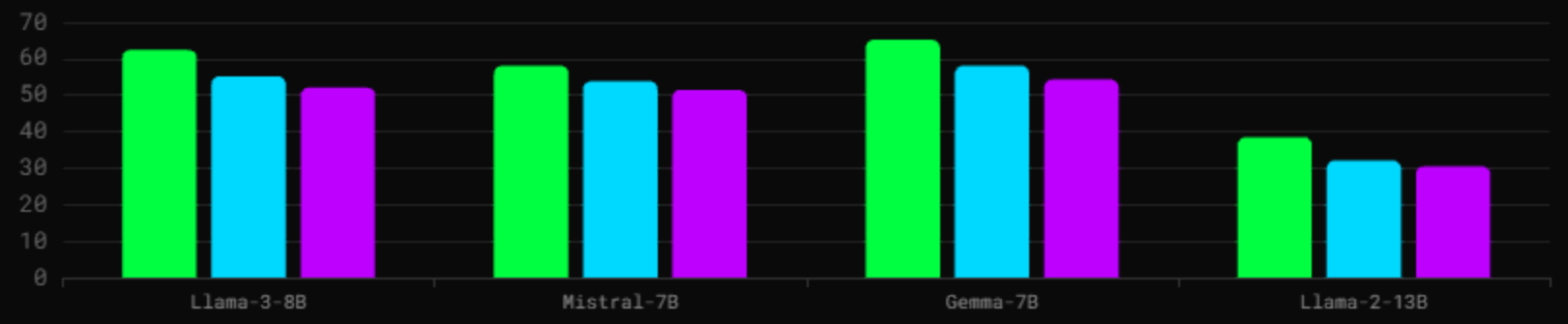
EFFICIENCY

4.8 GB

AVG VRAM USAGE

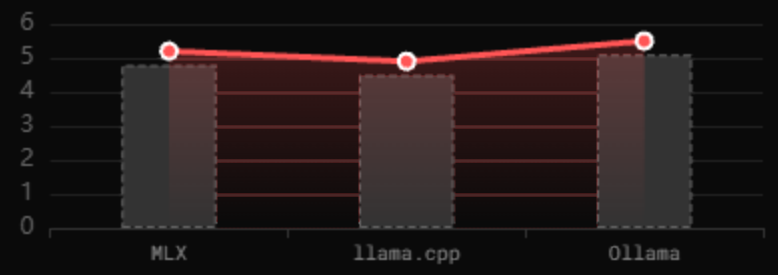
Idle overhead: ~150MB

Comparativo de Throughput (Tokens/s)



Consumo de VRAM (GB)

Llama-3-8B-4bit



Cenários de Execução

BACKEND	MODE	QUANT	SCORE
MLX	GPU (Metal)	4-bit	**98/100**
llama.cpp	CPU+GPU	Q4_K_M	**92/100**
Ollama	Auto	4-bit	**89/100**
llama.cpp	CPU Only	Q8_0	**45/100**

FINAL REMARKS

Conclusão do Projeto

STATUS: READY_FOR_DEPLOYMENT // PI_III_COMPLETE

FUNCTIONAL SOFTWARE

Plataforma Técnica Real

O MLX-Pilot não é apenas um conceito teórico. É um software funcional capaz de baixar, carregar e executar inferência de LLMs localmente com performance mensurável.

MODULAR ARCHITECTURE

Projeto Escalável

A arquitetura baseada em *Providers* e desacoplamento entre UI (Tauri) e Core (Rust) permite a fácil adição de novos backends e funcionalidades sem refatoração massiva.

PRODUCTION READY CORE

Arquitetura Robusta

O uso de Rust garante segurança de memória (memory safety) e alta performance, criando uma fundação sólida para transformar o protótipo acadêmico em um produto.

ACADEMIC VALUE

Laboratório de IA

Potencial para se tornar a ferramenta padrão da instituição para ensino de IA Generativa, permitindo que alunos experimentem sem custos de nuvem ou configurações complexas.

ACTION PLAN

Próximos Passos

SPRINT_STATUS: ACTIVE // T-MINUS 3 WEEKS

Consolidação

A fase final do Projeto Integrador III foca na estabilização da arquitetura e coleta de métricas para validação acadêmica. O objetivo é entregar um artefato robusto que sirva de base sólida para o TCC (PI IV).

PROJECT COMPLETION **85%**

START: FEB 2026 DEADLINE: JUN 2026

```
$ ./mlx-pilot --benchmark
Loading model weights... OK
Initializing Metal backend... OK
Running inference test [batch=1]...
Processing tokens...
```

Finalização do Benchmark

IN PROGRESS

Coleta sistemática de métricas de latência e consumo de VRAM em diferentes hardwares.

Implementação de Observabilidade

PENDING

Integração de dashboards para visualização de logs estruturados e tracing de requests.

Desenvolvimento do Agente

PLANNED

Design da arquitetura de raciocínio (Chain-of-Thought) para o assistente autônomo.

Preparação para PI IV

Q3 2026

Empacotamento, documentação técnica final e planejamento de entrega.

FINAL_REMARKS

Agradecimentos

ACKNOWLEDGEMENTS // CREDITS

ORIENTAÇÃO ACADÊMICA

Professores Orientadores

Pelo suporte técnico contínuo e direcionamento nas decisões de arquitetura de software e pesquisa.

INFRAESTRUTURA & APOIO

Instituição de Ensino

Pelo fornecimento do ambiente de laboratório e incentivo ao desenvolvimento tecnológico inovador.

ECOSSISTEMA

Comunidade Open-Source

Desenvolvedores do Rust, Tauri, MLX e llama.cpp, cujas bases tornaram este projeto possível.

POTENCIAL PARCERIA

NVIDIA

Pelas ferramentas CUDA e suporte ao ecossistema acadêmico de IA, fundamentais para nossos benchmarks futuros.

OPEN SOURCE

Contato e Repositório

JOIN_THE_REVOLUTION // CONTRIBUTE

- Seeking Maintainers
- Feedback Welcome

v0.1.0-alpha

LATEST RELEASE

mlx-pilot/core

Repositório oficial do projeto. Acesse o código fonte, documentação técnica, benchmarks e roadmap detalhado.

124	32	18
STARS	FORKS	ISSUES

- [DOCS] Architecture Overview
- [GUIDE] Setting up local environment
- [CONTRIB] Good first issues

SCAN FOR MOBILE

DIRECT_ACCESS

CORE TEAM

[Nome do Integrante 1]
LEAD_ARCHITECT

[Nome do Integrante 2]
AI_ENGINEER

[Nome do Integrante 3]
RUST_DEVELOPER

[Nome do Integrante 4]
QA_&_DEVOPS

Colaboração Institucional

Interessado em utilizar o MLX-Pilot em sua disciplina ou pesquisa? Entre em contato para suporte à implantação no laboratório.